증명 검증 프로세스 (Proof Verification Process)

개요

증명 검증(Proof Verification)은 증명자가 생성한 증명의 유효성을 검증하는 과정입니다. 검증자(Verifier)는 증명과 공개 입력만을 사용하여, 증명자가 올바른 witness를 알고 있으며 모든 회로 제약 조건을 만족함을 확인합니다.

Plonky2의 검증 과정은 증명 생성 과정을 역으로 추적하면서, 각 단계에서 생성된 커밋먼트와 챌린지를 재계산하고 검증합니다. 이 문서에서는 `verify` 함수의 전체 흐름을 단계별로 분석합니다.

1. 검증 메인 함수

1.1 verify 함수 시그니처

```
pub(crate) fn  verify<F: RichField + Extendable<D>, C: GenericConfig<D, F = F>,
proof_with_pis: ProofWithPublicInputs<F, C, D>,
verifier_data: &VerifierOnlyCircuitData<C, D>,
common_data: &CommonCircuitData<F, D>,
) -> Result<()>
```

입력 파라미터:

- `proof_with_pis` : 증명과 공개 입력을 포함하는 구조체
- `verifier_data` : 검증자 전용 데이터 (constants_sigmas_cap, circuit_digest)
- `common_data` : 증명자와 검증자가 공유하는 회로 데이터

반환값:

- `Result<()>` : 검증 성공 시 `Ok(())` , 실패 시 에러

1.2 ProofWithPublicInputs 구조

```
pub struct ProofWithPublicInputs<F: RichField + Extendable<D>, C: GenericConfig
pub proof: Proof<F, C, D>,
pub public_inputs: Vec<F>,
}
```

Proof 구조:

```
pub struct Proof<F: RichField + Extendable<D>, C: GenericConfig<D, F = F>, cons
/// Wire 다항식들의 Merkle Cap
pub wires_cap: MerkleCap<F, C::Hasher>,
/// Z와 Partial Products 다항식들의 Merkle Cap
pub plonk_zs_partial_products_cap: MerkleCap<F, C::Hasher>,
/// Quotient 다항식들의 Merkle Cap
pub quotient_polys_cap: MerkleCap<F, C::Hasher>,
/// 평가 점에서의 다항식 값들
pub openings: OpeningSet<F, D>,
/// FRI Opening Proof
pub opening_proof: FriProof<F, C::Hasher, D>,
```

```
}
```

2. 공개 입력 해시 검증

2.1 get_public_inputs_hash 함수

검증의 첫 단계는 공개 입력의 해시를 계산하는 것입니다.

```
let public_inputs_hash = proof_with_pis.get_public_inputs_hash();
```

내부 구현:

```
impl<F: RichField + Extendable<D>, C: GenericConfig<D, F = F>, const D: usize>
ProofWithPublicInputs<F, C, D>
{
pub fn get_public_inputs_hash(&self) -> <<C as GenericConfig<D>>::InnerHasher
hash_n_to_hash_no_pad::<F, C::InnerHasher>(&self.public_inputs)
}
}
```

동작 과정:

1. 모든 공개 입력 값을 해시 함수에 입력
2. Poseidon 해시 (또는 설정된 InnerHasher) 사용
3. 4개의 필드 원소로 구성된 해시 출력 생성

목적:

- 공개 입력의 무결성 보장
- 증명자가 올바른 공개 입력을 사용했는지 확인
- 회로 내 PublicInputGate와 연결

2.2 공개 입력 해시의 역할

증명 생성 시:

```
// Prover가 공개 입력 해시 계산
let  public_inputs_hash = hash_n_to_hash_no_pad::<F, C::InnerHasher>(&public_inp


// Challenger에 관찰
challenger.observe_hash::<C::InnerHasher>(public_inputs_hash);
```

검증 시:

```
// Verifier가 동일한 해시 재계산
let  public_inputs_hash = proof_with_pis.get_public_inputs_hash();


// 동일한 방식으로 Challenger에 관찰
challenger.observe_hash::<C::InnerHasher>(public_inputs_hash);
```

이를 통해 증명자와 검증자가 동일한 공개 입력을 사용함을 보장합니다.

3. 챌린지 재생성

3.1 get_challenges 함수

Fiat-Shamir 변환을 통해 증명 생성 시 사용된 모든 챌린지를 재생성합니다.

```
let challenges = proof_with_pis.get_challenges(
public_inputs_hash,
&verifier_data.circuit_digest,
common_data,
)?;
```

반환 구조:

```
pub struct ProofChallenges<F: RichField + Extendable<D>, const D: usize> {
/// Permutation argument용 Beta 챌린지
pub plonk_betas: Vec<F>,
/// Permutation argument용 Gamma 챌린지
pub plonk_gammas: Vec<F>,
/// 제약조건 결합용 Alpha 챌린지
pub plonk_alphas: Vec<F>,
/// Lookup argument용 Delta 챌린지 (있는 경우)
pub plonk_deltas: Vec<F>,
/// 다항식 평가 점 Zeta
pub plonk_zeta: F::Extension,
/// FRI 프로토콜 챌린지들
pub fri_challenges: FriChallenges<F, D>,
```

```
}
```

3.2 챌린지 재생성 과정

```
pub fn get_challenges(
&self,
public_inputs_hash: <<C as GenericConfig<D>>::InnerHasher as Hasher<F>>::Hash,
circuit_digest: &<<C as GenericConfig<D>>::Hasher as Hasher<F>>::Hash,
common_data: &CommonCircuitData<F, D>,
) -> Result<ProofChallenges<F, D>> {
let num_challenges = common_data.config.num_challenges;
let mut challenger = Challenger::<F, C::Hasher>::new();
// 1. FRI 파라미터 관찰
common_data.fri_params.observe(&mut challenger);
// 2. 회로 다이제스트 관찰
challenger.observe_hash::<C::Hasher>(*circuit_digest);
// 3. 공개 입력 해시 관찰
challenger.observe_hash::<C::InnerHasher>(public_inputs_hash);
// 4. Wire Cap 관찰
challenger.observe_cap::<C::Hasher>(&self.proof.wires_cap);
// 5. Beta, Gamma 챌린지 생성
let plonk_betas = challenger.get_n_challenges(num_challenges);
let plonk_gammas = challenger.get_n_challenges(num_challenges);
```

```
// 6. Lookup Delta 챌린지 생성 (필요한 경우)
let has_lookup = common_data.num_lookup_polys != 0;
let plonk_deltas = if has_lookup {
let num_lookup_challenges = common_data.num_lookup_challenges();
let mut deltas = Vec::with_capacity(2 * num_challenges);
let num_additional = num_lookup_challenges - 2 * num_challenges;
let additional = challenger.get_n_challenges(num_additional);
deltas.extend(&plonk_betas);
deltas.extend(&plonk_gammas);
deltas.extend(additional);
deltas
} else {
vec![]
};
// 7. Z와 Partial Products Cap 관찰
challenger.observe_cap::<C::Hasher>(&self.proof.plonk_zs_partial_products_cap);
// 8. Alpha 챌린지 생성
let plonk_alphas = challenger.get_n_challenges(num_challenges);
// 9. Quotient Cap 관찰
challenger.observe_cap::<C::Hasher>(&self.proof.quotient_polys_cap);
// 10. Zeta 챌린지 생성
let plonk_zeta = challenger.get_extension_challenge::<D>();
// 11. Opening Set 관찰
```

```
challenger.observe_openings(&self.proof.openings.to_fri_openings());

// 12. FRI 챌린지 생성

let  fri_challenges = challenger.fri_challenges::<C, D>(

&common_data.fri_params,

&self.proof.opening_proof.commit_phase_merkle_caps,

&self.proof.opening_proof.final_poly,

self.proof.opening_proof.pow_witness,

)?;

Ok(ProofChallenges {

plonk_betas,

plonk_gammas,

plonk_alphas,

plonk_deltas,

plonk_zeta,

fri_challenges,

})

}
```

중요 포인트:

- 증명자와 정확히 동일한 순서로 관찰해야 함
- 순서가 다르면 다른 챌린지가 생성되어 검증 실패
- 모든 커밋먼트(Cap)를 관찰하여 보안성 보장

4. Vanishing Polynomial 평가

4.1 eval_vanishing_poly 함수

Vanishing Polynomial은 모든 제약조건을 하나의 다항식으로 결합한 것입니다. 검증자는 이 다항식이 Zeta 점에서 올바른 값을 가지는지 확인합니다.

```
let  vanishing_polys_zeta = eval_vanishing_poly::<F, D>(
common_data,
challenges.plonk_zeta,
vars,
local_zs,
next_zs,
local_lookup_zs,
next_lookup_zs,
partial_products,
s_sigmas,
&challenges.plonk_betas,
&challenges.plonk_gammas,
&challenges.plonk_alphas,
&challenges.plonk_deltas,
);
```

입력 파라미터:

- `plonk_zeta` : 평가 점

- `vars` : Zeta에서의 Wire 값들
- `local_zs` , `next_zs` : Z 다항식의 현재/다음 값
- `local_lookup_zs` , `next_lookup_zs` : Lookup Z 다항식 값 (있는 경우)
- `partial_products` : Partial Products 값들
- `s_sigmas` : Sigma permutation 값들
- 챌린지들: Beta, Gamma, Alpha, Delta

4.2 EvaluationVars 구조

평가에 필요한 모든 변수를 담는 구조체입니다:

```
pub struct EvaluationVars<F: Field, const D: usize> {
/// 현재 행의 상수 값들
pub local_constants: &[F::Extension],
/// 현재 행의 Wire 값들
pub local_wires: &[F::Extension],
/// 공개 입력 해시
pub public_inputs_hash: &HashOut<F>,
}
```

변수 준비:

```
let vars = EvaluationVars {
local_constants: &proof.openings.constants,
local_wires: &proof.openings.wires,
public_inputs_hash: &public_inputs_hash,
```

```
};
```

4.3 제약조건 평가 과정

`eval_vanishing_poly` 함수는 세 가지 주요 제약조건을 평가합니다:

4.3.1 Gate 제약조건

각 게이트의 산술 제약조건을 평가합니다:

```
// 각 게이트 타입에 대해
for (i, gate) in  common_data.gates.iter().enumerate() {
let  selector_index = common_data.selectors_info.selector_indices[i];
// Selector 값 가져오기
let  selector = vars.local_constants[selector_index];
// 게이트 제약조건 평가
let  gate_constraints = gate.eval_unfiltered(vars);
// Alpha 챌린지로 결합
for (j, constraint) in  gate_constraints.iter().enumerate() {
let  alpha_pow = alphas[constraint_index];
vanishing_poly += alpha_pow * selector * constraint;
constraint_index += 1;
}
}
```

Selector의 역할:

- 특정 행에서 어떤 게이트가 활성화되는지 나타냄
- Selector가 1이면 해당 게이트 활성화, 0이면 비활성화
- 여러 게이트 타입을 하나의 회로에서 사용 가능

4.3.2 Permutation 제약조건

Copy constraint를 검증하는 Z 다항식 제약조건입니다:

```
// Z(1) = 1 제약
let z_first_constraint = local_zs[0] - F::Extension::ONE;
vanishing_poly += alphas[constraint_index] * z_first_constraint;
constraint_index += 1;


// Z 다항식 재귀 관계 제약
for i in 0..num_challenges {
let beta = betas[i];
let gamma = gammas[i];
// 분자 계산: Π(wire + β·s_id + γ)
let mut numerator = F::Extension::ONE;
for j in 0..num_routed_wires {
let wire_value = vars.local_wires[j];
let k_i = k_is[j];
let s_id = k_i * zeta;
numerator *= wire_value + beta * s_id + gamma;
}
```

```
// 분모 계산: Π(wire + β·s_σ + γ)
let mut denominator = F::Extension::ONE;
for j in 0..num_routed_wires {
let wire_value = vars.local_wires[j];
let s_sigma = s_sigmas[j];
denominator *= wire_value + beta * s_sigma + gamma;
}
// Z(gζ) = Z(ζ) · (분자/분모) 제약
let z_constraint = next_zs[i] * denominator - local_zs[i] * numerator;
vanishing_poly += alphas[constraint_index] * z_constraint;
constraint_index += 1;
}
```

수학적 의미:

```
Z(g·ζ) = Z(ζ) · Π[j] (w_j(ζ) + β·s_id_j(ζ) + γ) / (w_j(ζ) + β·s_σ_j(ζ) + γ)
```

이 관계가 성립하면 Wire 값들이 올바른 순열을 따름을 의미합니다.

4.3.3 Lookup 제약조건 (선택적)

Lookup 테이블이 있는 경우 추가 제약조건을 평가합니다:

```
if has_lookup {
// Lookup Z 다항식 제약조건
for i in 0..num_lookup_challenges {
```

```
let delta = deltas[i];
// Lookup 관련 제약조건 평가
let lookup_constraint = evaluate_lookup_constraint(
vars,
local_lookup_zs[i],
next_lookup_zs[i],
delta,
lookup_tables,
);
vanishing_poly += alphas[constraint_index] * lookup_constraint;
constraint_index += 1;
}
}
```

Lookup의 목적:

- Range check 등의 연산을 효율적으로 구현
- 미리 정의된 테이블에서 값 조회
- LogUp argument를 사용하여 검증

4.4 반환값

`eval_vanishing_poly` 는 각 챌린지에 대한 Vanishing Polynomial 값을 반환합니다:

```
pub fn eval_vanishing_poly<F: RichField + Extendable<D>, const D: usize>(
// ... 파라미터들
```

```
) -> Vec<F::Extension> {
let num_challenges = common_data.config.num_challenges;
let mut vanishing_polys = vec![F::Extension::ZERO; num_challenges];
// 각 챌린지에 대해 독립적으로 계산
for i in 0..num_challenges {
vanishing_polys[i] = evaluate_constraints_for_challenge_i(
// ... 파라미터들
i,
);
}
vanishing_polys
}
```

다중 챌린지의 이유:

- 보안성 향상 (soundness amplification)
- 각 챌린지는 독립적인 검증 제공
- 일반적으로 2-4개의 챌린지 사용

5. Quotient 검증 로직

5.1 Quotient Polynomial의 역할

Quotient Polynomial Q(x)는 다음 관계를 만족해야 합니다:

```
vanishing_poly(x) = Z_H(x) · Q(x)
```

여기서:

- `vanishing_poly(x)` : 모든 제약조건의 결합
- `Z_H(x) = x^n - 1` : Vanishing Polynomial (부분군 H에서 0)
- `Q(x)` : Quotient Polynomial

5.2 Quotient 재구성

증명에는 Q(x)가 청크로 분할되어 있습니다:

```
Q(x) = Q_0(x) + x^n · Q_1(x) + x^(2n) · Q_2(x) + ...
```

검증자는 Zeta에서 Q(ζ)를 재구성합니다:

```
// Zeta의 거듭제곱 계산
let zeta_pow_deg = challenges.plonk_zeta.exp_power_of_2(common_data.degree_bits

// Quotient 청크들을 결합
let quotient_polys_zeta: Vec<F::Extension> = proof.openings.quotient_polys
.chunks(common_data.quotient_degree_factor)
.map(|chunk| {
reduce_with_powers(chunk, zeta_pow_deg)
})
.collect();
```

reduce_with_powers 함수:

```
pub fn reduce_with_powers<F: Field>(terms: &[F], scalar: F) -> F {
let mut sum = F::ZERO;
let mut current = F::ONE;
for &term in terms {
sum += current * term;
current *= scalar;
}
sum
}
```

예시:

```
chunk = [Q_0(ζ), Q_1(ζ), Q_2(ζ)]
scalar = ζ^n

결과 = Q_0(ζ) + ζ^n · Q_1(ζ) + ζ^(2n) · Q_2(ζ)
```

5.3 Z_H(ζ) 계산

Vanishing Polynomial을 Zeta에서 평가합니다:

```
let z_h_zeta = challenges.plonk_zeta.exp_power_of_2(common_data.degree_bits())
- F::Extension::ONE;
```

수학적 의미:

```
Z_H(ζ) = ζ^n - 1
```

여기서 n은 회로의 차수(degree)입니다.

중요 검증:

```
// Zeta가 부분군에 속하지 않는지 확인
ensure!(
z_h_zeta != F::Extension::ZERO,
"Opening point is in the subgroup."
);
```

만약 ζ ∈ H이면 Z_H(ζ) = 0이 되어 나눗셈이 불가능합니다.

5.4 Quotient 검증

각 챌린지에 대해 다음을 검증합니다:

```
for (i, &vanishing_poly_zeta) in  vanishing_polys_zeta.iter().enumerate() {
let  quotient_zeta = quotient_polys_zeta[i];
// vanishing_poly(ζ) = Z_H(ζ) · Q(ζ) 검증
let  expected = z_h_zeta * quotient_zeta;
ensure!(
vanishing_poly_zeta == expected,
"Quotient polynomial verification failed for challenge {}. \
Expected {}, got {}",
```

```
i,
expected,
vanishing_poly_zeta
);
}
```

검증 의미:

- 모든 제약조건이 부분군 H에서 만족됨을 확인
- Quotient가 올바르게 계산되었음을 검증
- 증명자가 정직하게 증명을 생성했음을 보장

6. FRI 검증 호출

6.1 verify_fri_proof 함수

FRI (Fast Reed-Solomon IOP) 검증은 다항식이 낮은 차수임을 확인합니다.

```
verify_fri_proof::<F, C, D>(
&common_data.get_fri_instance(challenges.plonk_zeta),
&proof.openings.to_fri_openings(),
&challenges.fri_challenges,
merkle_caps,
&proof.opening_proof,
&common_data.fri_params,
)?;
```

입력 파라미터:

- `fri_instance` : FRI 인스턴스 정보 (평가 점, 다항식 정보)
- `openings` : 주장된 다항식 평가값들
- `fri_challenges` : FRI 프로토콜 챌린지들
- `merkle_caps` : 모든 다항식 배치의 Merkle Cap들
- `opening_proof` : FRI Opening Proof
- `fri_params` : FRI 프로토콜 파라미터

6.2 FRI Instance 구성

```
pub fn get_fri_instance(&self, zeta: F::Extension) -> FriInstanceInfo<F, D> {
let g = F::Extension::primitive_root_of_unity(self.degree_bits());
let zeta_next = g * zeta;
// Zeta에서 평가할 다항식들
let zeta_batch = FriBatchInfo {
point: zeta,
polynomials: vec![
FriPolynomialInfo { oracle_index: 0, polynomial_index: 0 }, // constants
FriPolynomialInfo { oracle_index: 0, polynomial_index: 1 }, // sigmas
FriPolynomialInfo { oracle_index: 1, polynomial_index: 0 }, // wires
FriPolynomialInfo { oracle_index: 2, polynomial_index: 0 }, // zs
FriPolynomialInfo { oracle_index: 2, polynomial_index: 1 }, // partial_products
FriPolynomialInfo { oracle_index: 3, polynomial_index: 0 }, // quotient
// ... 기타 다항식들
```

```
],
};
// g·Zeta에서 평가할 다항식들 (Z 다항식의 "다음" 값)
let  zeta_next_batch = FriBatchInfo {
point: zeta_next,
polynomials: vec![
FriPolynomialInfo { oracle_index: 2, polynomial_index: 0 }, // zs_next
// ... 기타 "next" 다항식들
],
};
FriInstanceInfo {
oracles: vec![
FriOracleInfo { num_polys: 2, blinding: false }, // constants_sigmas
FriOracleInfo { num_polys: 1, blinding: true }, // wires
FriOracleInfo { num_polys: 2, blinding: true }, // zs_partial_products
FriOracleInfo { num_polys: 1, blinding: true }, // quotient
],
batches: vec![zeta_batch, zeta_next_batch],
}
}
```

6.3 FRI 검증 과정

FRI 검증은 다음 단계로 진행됩니다:

6.3.1 초기 다항식 결합

모든 다항식을 하나의 다항식으로 결합합니다:

```
// 각 배치에 대해
for batch in instance.batches {
let point = batch.point;
// 배치 내 모든 다항식을 랜덤 선형 결합
let mut combined = F::Extension::ZERO;
for poly_info in batch.polynomials {
let opening = openings[poly_info.oracle_index][poly_info.polynomial_index];
let alpha = fri_challenges.fri_alpha.pow(poly_index);
combined += alpha * opening;
poly_index += 1;
}
// 결합된 다항식을 평가 점에서 shift
// combined_poly(x) - combined_value
// 이 다항식은 (x - point)로 나누어떨어져야 함
}
```

6.3.2 Commit Phase 검증

FRI Commit Phase에서 생성된 Merkle Tree들을 검증합니다:

```
for (i, cap) in opening_proof.commit_phase_merkle_caps.iter().enumerate() {
```

```
// 각 라운드의 Cap 관찰
challenger.observe_cap(cap);
// Beta 챌린지 생성
let beta = challenger.get_extension_challenge();
// 다항식 폴딩 검증
// f(x) → f_folded(x^2) = f_even(x^2) + β · f_odd(x^2)
}
```

6.3.3 Query Phase 검증

랜덤 인덱스에서 다항식 값과 Merkle Proof를 검증합니다:

```
// 쿼리 인덱스 생성
let query_indices = get_fri_query_indices(
&fri_challenges,
common_data.degree_bits(),
fri_params.num_query_rounds,
);

for &query_index in &query_indices {
// 각 쿼리에 대해
let query_proof = &opening_proof.query_round_proofs[query_index];
// 1. 초기 다항식 값 검증
verify_initial_proof(query_proof, merkle_caps, query_index)?;
```

```
// 2. 폴딩 일관성 검증
verify_folding_consistency(query_proof, fri_challenges)?;
// 3. Merkle Proof 검증
for step in &query_proof.steps {
verify_merkle_proof(
step.merkle_proof,
step.cap,
query_index,
step.values,
)?;
}
}
```

6.3.4 Final Polynomial 검증

FRI의 마지막 단계에서 다항식이 충분히 작은 차수로 축소되었는지 확인합니다:

```
// Final Polynomial의 차수 확인
let final_poly_len = opening_proof.final_poly.len();
ensure!(
final_poly_len <= fri_params.final_poly_len(),
"Final polynomial degree too high: {} > {}",
final_poly_len,
fri_params.final_poly_len()
);
```

```
// Final Polynomial 관찰
challenger.observe_extension_elements(&opening_proof.final_poly);


// Proof of Work 검증 (선택적)
if  fri_params.proof_of_work_bits > 0 {
verify_proof_of_work(
challenger,
opening_proof.pow_witness,
fri_params.proof_of_work_bits,
)?;
}
```

Final Polynomial의 의미:

- 반복적인 폴딩 후 남은 작은 차수의 다항식
- 직접 계수를 전송하여 검증
- 차수가 충분히 작으면 다항식이 low-degree임을 보장

6.4 FRI 검증의 보안성

FRI 검증은 다음을 보장합니다:

1. **Low Degree Test**: 다항식이 주장된 차수 이하임
2. **Proximity Test**: 커밋된 값들이 실제 다항식의 평가값에 가까움
3. **Soundness**: 증명자가 거짓 증명을 생성할 확률이 매우 낮음

보안 파라미터:

- `rate_bits` : 코드 레이트 (낮을수록 안전)
- `num_query_rounds` : 쿼리 횟수 (많을수록 안전)
- `proof_of_work_bits` : 작업 증명 비트 (높을수록 안전)

Soundness 계산:

```
soundness_bits = min(
field_bits,
num_query_rounds * rate_bits + proof_of_work_bits
)
```

7. 검증 전체 흐름 요약

검증 과정을 단계별로 정리하면 다음과 같습니다:

```
1. 공개 입력 해시 계산
└─ hash_n_to_hash_no_pad(public_inputs)


2. 챌린지 재생성
├─ Challenger 초기화
├─ FRI 파라미터 관찰
├─ 회로 다이제스트 관찰
├─ 공개 입력 해시 관찰
├─ Wire Cap 관찰 → Beta, Gamma 생성
```

├─ Z Cap 관찰 → Alpha 생성

├─ Quotient Cap 관찰 → Zeta 생성

└─ Opening Set 관찰 → FRI 챌린지 생성

3. Vanishing Polynomial 평가

├─ Gate 제약조건 평가

├─ Permutation 제약조건 평가

└─ Lookup 제약조건 평가 (선택적)

4. Quotient 검증

├─ Quotient 청크 재구성

├─ Z_H(ζ) 계산

└─ vanishing_poly(ζ) = Z_H(ζ) · Q(ζ) 검증

5. FRI 검증

├─ 초기 다항식 결합

├─ Commit Phase 검증

├─ Query Phase 검증

└─ Final Polynomial 검증

6. 검증 완료

└─ Ok(()) 반환

8. 코드 예시: 전체 검증 흐름

```rust
use anyhow::Result;

use plonky2::plonk::config::{GenericConfig, PoseidonGoldilocksConfig};

use plonky2::plonk::proof::ProofWithPublicInputs;

use plonky2::plonk::circuit_data::{VerifierOnlyCircuitData, CommonCircuitData};


fn verify_proof<F, C, const D: usize>(

proof_with_pis: ProofWithPublicInputs<F, C, D>,

verifier_data: &VerifierOnlyCircuitData<C, D>,

common_data: &CommonCircuitData<F, D>,

) -> Result<()>

where

F: RichField + Extendable<D>,

C: GenericConfig<D, F = F>,

{

// 1. 공개 입력 해시 계산

let public_inputs_hash = proof_with_pis.get_public_inputs_hash();

// 2. 챌린지 재생성

let challenges = proof_with_pis.get_challenges(

public_inputs_hash,

&verifier_data.circuit_digest,

common_data,
```

```
)?;

// 3. Opening Set에서 변수 추출

let  vars = EvaluationVars {

local_constants: &proof_with_pis.proof.openings.constants,

local_wires: &proof_with_pis.proof.openings.wires,

public_inputs_hash: &public_inputs_hash,

};

let  local_zs = &proof_with_pis.proof.openings.plonk_zs;

let  next_zs = &proof_with_pis.proof.openings.plonk_zs_next;

let  partial_products = &proof_with_pis.proof.openings.partial_products;

let  s_sigmas = &proof_with_pis.proof.openings.plonk_sigmas;

// 4. Vanishing Polynomial 평가

let  vanishing_polys_zeta = eval_vanishing_poly::<F, D>(

common_data,

challenges.plonk_zeta,

vars,

local_zs,

next_zs,

&[], // local_lookup_zs (없는 경우)

&[], // next_lookup_zs (없는 경우)

partial_products,

s_sigmas,

&challenges.plonk_betas,
```

```
&challenges.plonk_gammas,
&challenges.plonk_alphas,
&challenges.plonk_deltas,
);
// 5. Z_H(ζ) 계산
let  zeta_pow_deg = challenges.plonk_zeta
.exp_power_of_2(common_data.degree_bits());
let  z_h_zeta = zeta_pow_deg - F::Extension::ONE;
ensure!(
z_h_zeta != F::Extension::ZERO,
"Opening point is in the subgroup"
);
// 6. Quotient 재구성 및 검증
let  quotient_polys_zeta: Vec<_> = proof_with_pis.proof.openings.quotient_polys
.chunks(common_data.quotient_degree_factor)
.map(|chunk| reduce_with_powers(chunk, zeta_pow_deg))
.collect();
for (i, (&vanishing_poly_zeta, &quotient_zeta)) in
vanishing_polys_zeta.iter()
.zip(quotient_polys_zeta.iter())
.enumerate()
{
let  expected = z_h_zeta * quotient_zeta;
```

```
ensure!(
vanishing_poly_zeta == expected,
"Quotient verification failed for challenge {}", i
);
}
// 7. FRI 검증
let  merkle_caps = vec![
verifier_data.constants_sigmas_cap.clone(),
proof_with_pis.proof.wires_cap.clone(),
proof_with_pis.proof.plonk_zs_partial_products_cap.clone(),
proof_with_pis.proof.quotient_polys_cap.clone(),
];
verify_fri_proof::<F, C, D>(
&common_data.get_fri_instance(challenges.plonk_zeta),
&proof_with_pis.proof.openings.to_fri_openings(),
&challenges.fri_challenges,
merkle_caps,
&proof_with_pis.proof.opening_proof,
&common_data.fri_params,
)?;
Ok(())
}
```

9. 검증 실패 시나리오

검증이 실패할 수 있는 주요 경우들:

9.1 공개 입력 불일치

```
// 증명 생성 시 사용한 공개 입력과 검증 시 제공된 공개 입력이 다른 경우
let proof_public_inputs = vec![F::from_canonical_u64(10)];
let verify_public_inputs = vec![F::from_canonical_u64(20)];

// 공개 입력 해시가 달라져 챌린지가 다르게 생성됨
// → 모든 후속 검증 실패
```

9.2 제약조건 위반

```
// 회로에서 a + b = c를 요구했지만
// witness에서 a = 3, b = 4, c = 8로 설정한 경우

// Vanishing Polynomial 평가 시 0이 아닌 값 발생
// → Quotient 검증 실패
```

9.3 Permutation 위반

```
// connect(x, y)로 연결했지만
```

```
// witness에서 x = 5, y = 7로 다른 값 설정


// Z 다항식 재귀 관계가 성립하지 않음
// → Permutation 제약조건 검증 실패
```

9.4 잘못된 다항식 차수

```
// 증명자가 높은 차수의 다항식을 사용한 경우
// FRI Query Phase에서 일관성 검증 실패
// → FRI 검증 실패
```

9.5 Merkle Proof 위조

```
// 증명자가 커밋먼트와 다른 값을 Opening에 제공한 경우
// Merkle Proof 검증 실패
// → FRI 검증 실패
```

10. 성능 고려사항

10.1 검증 시간 분석

검증 시간의 주요 구성 요소:

1. **챌린지 재생성**: O(1) - 해시 연산
2. **Vanishing Polynomial 평가**: O(num_gates) - 선형 시간

3. **Quotient 검증**: O(num_challenges) - 상수 시간

4. **FRI 검증**: O(num_query_rounds · log(degree)) - 로그 시간

전체 복잡도: O(num_gates + num_query_rounds · log(degree))

10.2 최적화 기법

```
// 1. 배치 역원 계산
let denominators = compute_all_denominators();
let inverses = F::batch_multiplicative_inverse(&denominators);

// 2. 사전 계산된 상수 재사용
let zeta_pow_deg = challenges.plonk_zeta.exp_power_of_2(degree_bits);
// 여러 곳에서 재사용

// 3. 병렬 처리 (가능한 경우)
vanishing_polys.par_iter()
.enumerate()
.for_each(|(i, &vanishing_poly)| {
verify_quotient(i, vanishing_poly, quotient_polys[i]);
});
```

10.3 증명 크기

Plonky2 증명의 크기 구성:

- **Merkle Caps**: 3개 × cap_height × hash_size

- **Opening Set**: 약 20-30개의 필드 원소
- **FRI Proof**:
- Commit Phase Caps: num_rounds × cap_height × hash_size
- Query Proofs: num_query_rounds × log(degree) × hash_size
- Final Polynomial: final_poly_len × field_size

일반적인 크기: 약 45-100 KB (설정에 따라 다름)

11. 검증자 데이터 구조

11.1 VerifierOnlyCircuitData

검증자만 필요한 데이터:

```
pub struct VerifierOnlyCircuitData<C: GenericConfig<D>, const D: usize> {
/// Constants와 Sigma 다항식의 Merkle Cap
pub constants_sigmas_cap: MerkleCap<F, C::Hasher>,
/// 회로의 해시 다이제스트
pub circuit_digest: HashOut<F>,
}
```

constants_sigmas_cap의 역할:

- 회로의 구조를 나타내는 상수들의 커밋먼트
- Permutation을 정의하는 Sigma 다항식의 커밋먼트
- 증명자가 올바른 회로를 사용했는지 확인

circuit_digest의 역할:

- 전체 회로의 고유 식별자
- 회로 구조, 게이트 구성, 설정 등을 해시
- 증명과 회로의 바인딩 보장

11.2 CommonCircuitData

증명자와 검증자가 공유하는 데이터:

```
pub struct CommonCircuitData<F: RichField + Extendable<D>, const D: usize> {
/// 회로 설정
pub config: CircuitConfig,
/// FRI 프로토콜 파라미터
pub fri_params: FriParams,
/// 사용된 게이트 타입들
pub gates: Vec<GateRef<F, D>>,
/// Selector 정보
pub selectors_info: SelectorsInfo,
/// 회로 차수의 log2
pub degree_bits: usize,
/// Quotient 차수 계수
pub quotient_degree_factor: usize,
/// 라우팅 가능한 와이어 수
pub num_routed_wires: usize,
// ... 기타 필드들
```

```
}
```

12. 주요 개념 정리

12.1 Fiat-Shamir 변환

대화형 프로토콜:

```
Prover → Commitment → Verifier

Prover ← Challenge ← Verifier

Prover → Response → Verifier
```

비대화형 프로토콜 (Fiat-Shamir):

```
Prover → Commitment → Hash

Prover ← Challenge ← Hash(Commitment)

Prover → Response → (증명에 포함)


Verifier: 동일한 해시 과정으로 챌린지 재생성
```

12.2 Soundness vs Completeness

Completeness (완전성):

- 정직한 증명자는 항상 검증을 통과
- 올바른 witness가 있으면 증명 생성 가능

Soundness (건전성):

- 거짓 증명자는 높은 확률로 검증 실패
- 잘못된 witness로는 증명 생성 불가능

Plonky2의 보안 수준:

```
soundness_bits ≈ 100 bits (기본 설정)

→ 성공 확률 < 2^(-100)
```

12.3 Zero-Knowledge 속성

검증 과정에서 witness 정보가 누출되지 않음:

1. **Blinding**: 랜덤 다항식 추가
2. **Commitment**: Merkle Tree로 값 숨김
3. **Limited Opening**: 필요한 점에서만 평가값 공개

검증자는 다음만 알 수 있음:

- 공개 입력
- 증명이 유효함
- 회로 구조

검증자가 알 수 없는 것:

- 비공개 입력 (witness)
- 중간 계산 값들
- 구체적인 와이어 값들

13. 실전 예제: Fibonacci 검증

Fibonacci 회로의 증명을 검증하는 완전한 예제:

```rust
use anyhow::Result;
use plonky2::field::types::Field;
use plonky2::iop::witness::{PartialWitness, WitnessWrite};
use plonky2::plonk::circuit_builder::CircuitBuilder;
use plonky2::plonk::circuit_data::CircuitConfig;
use plonky2::plonk::config::{GenericConfig, PoseidonGoldilocksConfig};

fn main() -> Result<()> {
const D: usize = 2;
type C = PoseidonGoldilocksConfig;
type F = <C as GenericConfig<D>>::F;

// 1. 회로 구성
let config = CircuitConfig::standard_recursion_config();
let mut builder = CircuitBuilder::<F, D>::new(config);

let initial_a = builder.add_virtual_target();
let initial_b = builder.add_virtual_target();

let mut prev_target = initial_a;
let mut cur_target = initial_b;
for _ in 0..99 {
```

```
let temp = builder.add(prev_target, cur_target);
prev_target = cur_target;
cur_target = temp;
}

builder.register_public_input(initial_a);
builder.register_public_input(initial_b);
builder.register_public_input(cur_target);

let data = builder.build::<C>();

// 2. 증명 생성
let mut pw = PartialWitness::new();
pw.set_target(initial_a, F::ZERO)?;
pw.set_target(initial_b, F::ONE)?;

println!("Generating proof...");
let proof = data.prove(pw)?;
println!("Proof generated successfully!");
println!("Public inputs: {:?}", proof.public_inputs);
println!("Proof size: {} bytes",
std::mem::size_of_val(&proof.proof));
```

```rust
// 3. 증명 검증
println!("\nVerifying proof...");
// 검증 시작 시간 측정
let start = std::time::Instant::now();
// 실제 검증 수행
data.verify(proof.clone())?;
let duration = start.elapsed();
println!("Verification successful!");
println!("Verification time: {:?}", duration);


// 4. 검증 세부 정보 출력
println!("\n=== Verification Details ===");
println!("Circuit degree: 2^{}", data.common.degree_bits);
println!("Number of gates: {}", 1 << data.common.degree_bits);
println!("Number of public inputs: {}", proof.public_inputs.len());
println!("FRI queries: {}", data.common.fri_params.num_query_rounds);
println!("Security bits: ~{}", data.common.config.security_bits);


// 5. 잘못된 증명 검증 시도 (실패해야 함)
println!("\n=== Testing Invalid Proof ===");
let mut invalid_proof = proof.clone();
invalid_proof.public_inputs[2] = F::from_canonical_u64(12345); // 잘못된 값
match data.verify(invalid_proof) {
```

```
Ok(_) => println!("ERROR: Invalid proof was accepted!"),
Err(e) => println!("Invalid proof correctly rejected: {}", e),
}

Ok(())
}
```

실행 결과 예시:

```
Generating proof...
Proof generated successfully!
Public inputs: [0, 1, 218922995834555169026]
Proof size: 87432 bytes

Verifying proof...
Verification successful!
Verification time: 12.5ms

=== Verification Details ===
Circuit degree: 2^12
Number of gates: 4096
Number of public inputs: 3
FRI queries: 28
Security bits: ~100
```

```
=== Testing Invalid Proof ===

Invalid proof correctly rejected: Quotient verification failed for challenge 0
```

14. 디버깅 가이드

14.1 일반적인 검증 오류

오류 1: "Opening point is in the subgroup"

```
// 원인: Zeta가 부분군 H에 속함

// 해결: Challenger의 관찰 순서 확인, 충분한 엔트로피 보장
```

오류 2: "Quotient verification failed"

```
// 원인: 제약조건 위반 또는 잘못된 witness

// 디버깅:

println!("Vanishing poly at zeta: {:?}", vanishing_poly_zeta);

println!("Expected (Z_H * Q): {:?}", z_h_zeta * quotient_zeta);

println!("Difference: {:?}", vanishing_poly_zeta - z_h_zeta * quotient_zeta);
```

오류 3: "FRI verification failed"

```
// 원인: 다항식 차수 문제 또는 Merkle Proof 불일치

// 디버깅:
```

```
// 1. 각 FRI 라운드의 차수 확인
// 2. Query 인덱스와 Merkle Proof 일치 여부 확인
// 3. 폴딩 일관성 검증
```

14.2 검증 로깅

상세한 검증 로그를 위한 헬퍼 함수:

```
fn verify_with_logging<F, C, const D: usize>(
proof_with_pis: ProofWithPublicInputs<F, C, D>,
verifier_data: &VerifierOnlyCircuitData<C, D>,
common_data: &CommonCircuitData<F, D>,
) -> Result<()>
where
F: RichField + Extendable<D>,
C: GenericConfig<D, F = F>,
{
println!("=== Starting Verification ===");
// 1. 공개 입력 해시
println!("Step 1: Computing public inputs hash...");
let public_inputs_hash = proof_with_pis.get_public_inputs_hash();
println!(" Hash: {:?}", public_inputs_hash);
// 2. 챌린지 재생성
println!("\nStep 2: Regenerating challenges...");
```

```
let  challenges = proof_with_pis.get_challenges(
public_inputs_hash,
&verifier_data.circuit_digest,
common_data,
)?;
println!(" Betas: {:?}", challenges.plonk_betas);
println!(" Gammas: {:?}", challenges.plonk_gammas);
println!(" Alphas: {:?}", challenges.plonk_alphas);
println!(" Zeta: {:?}", challenges.plonk_zeta);
// 3. Vanishing Polynomial 평가
println!("\nStep 3: Evaluating vanishing polynomial...");
let  vars = EvaluationVars {
local_constants: &proof_with_pis.proof.openings.constants,
local_wires: &proof_with_pis.proof.openings.wires,
public_inputs_hash: &public_inputs_hash,
};
let  vanishing_polys_zeta = eval_vanishing_poly::<F, D>(
common_data,
challenges.plonk_zeta,
vars,
&proof_with_pis.proof.openings.plonk_zs,
&proof_with_pis.proof.openings.plonk_zs_next,
&[],
```

```
&[],
&proof_with_pis.proof.openings.partial_products,
&proof_with_pis.proof.openings.plonk_sigmas,
&challenges.plonk_betas,
&challenges.plonk_gammas,
&challenges.plonk_alphas,
&challenges.plonk_deltas,
);
println!(" Vanishing polys: {:?}", vanishing_polys_zeta);
// 4. Quotient 검증
println!("\nStep 4: Verifying quotient polynomial...");
let  zeta_pow_deg = challenges.plonk_zeta
.exp_power_of_2(common_data.degree_bits());
let  z_h_zeta = zeta_pow_deg - F::Extension::ONE;
println!(" Z_H(zeta): {:?}", z_h_zeta);
let  quotient_polys_zeta: Vec<_> = proof_with_pis.proof.openings.quotient_polys
.chunks(common_data.quotient_degree_factor)
.map(|chunk| reduce_with_powers(chunk, zeta_pow_deg))
.collect();
for (i, (&vanishing, &quotient)) in  vanishing_polys_zeta.iter()
.zip(quotient_polys_zeta.iter())
.enumerate()
{
```

```
let  expected = z_h_zeta * quotient;

println!(" Challenge {}: vanishing={:?}, expected={:?}, match={}",

i, vanishing, expected, vanishing == expected);

ensure!(vanishing == expected, "Quotient verification failed");

}

// 5. FRI 검증

println!("\nStep 5: Verifying FRI proof...");

let  merkle_caps = vec![

verifier_data.constants_sigmas_cap.clone(),

proof_with_pis.proof.wires_cap.clone(),

proof_with_pis.proof.plonk_zs_partial_products_cap.clone(),

proof_with_pis.proof.quotient_polys_cap.clone(),

];

verify_fri_proof::<F, C, D>(

&common_data.get_fri_instance(challenges.plonk_zeta),

&proof_with_pis.proof.openings.to_fri_openings(),

&challenges.fri_challenges,

merkle_caps,

&proof_with_pis.proof.opening_proof,

&common_data.fri_params,

)?;

println!(" FRI verification passed!");

println!("\n=== Verification Complete ===");
```

```
Ok(())
}
```

15. 다음 단계

검증 프로세스를 이해했다면 다음 주제로 진행할 수 있습니다:

1. **FRI 프로토콜 상세**: 05-fri-protocol.md

- FRI Commit Phase 분석
- FRI Query Phase 분석
- 폴딩 메커니즘 이해

2. **Goldilocks Field**: 06-goldilocks-field.md

- 필드 연산 구현
- 최적화 기법
- SNARK에 적합한 이유

3. **재귀 증명**: 10-recursive-proofs.md

- 증명의 증명
- 재귀 검증 회로
- 증명 압축

16. 참고 자료

16.1 관련 문서

- 01-architecture.md - Plonky2 전체 아키텍처
- 02-circuit-building.md - 회로 구성 프로세스

- 03-proof-generation.md - 증명 생성 프로세스

16.2 소스 코드 참조

- `plonky2/src/plonk/verifier.rs` - 검증 메인 로직
- `plonky2/src/plonk/vanishing_poly.rs` - Vanishing Polynomial 평가
- `plonky2/src/fri/verifier.rs` - FRI 검증
- `plonky2/src/hash/merkle_proofs.rs` - Merkle Proof 검증

16.3 학술 자료

- **PLONK 논문**: "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge"
- **FRI 논문**: "Fast Reed-Solomon Interactive Oracle Proofs of Proximity"
- **Fiat-Shamir**: "How to Prove Yourself: Practical Solutions to Identification and Signature Problems"

요약

증명 검증은 다음 핵심 단계로 구성됩니다:

1. **공개 입력 해시 계산**: 공개 입력의 무결성 보장
2. **챌린지 재생성**: Fiat-Shamir 변환으로 모든 챌린지 재계산
3. **Vanishing Polynomial 평가**: 모든 제약조건을 Zeta에서 평가
4. **Quotient 검증**: vanishing_poly(ζ) = Z_H(ζ) · Q(ζ) 확인
5. **FRI 검증**: 다항식의 low-degree 속성 검증

검증 과정은 증명 생성의 역순으로 진행되며, 각 단계에서 증명자가 제공한 값들의 일관성과 정확성을 확인합니다. Plonky2의 검증은 매우 효율적이며, 일반적으로 수 밀리초 내에 완료됩니다.

검증자는 증명과 공개 입력만으로 계산의 정확성을 확인할 수 있으며, 비공개 입력(witness)에 대한 정보는 전혀 얻을 수 없습니다. 이것이 바로 영지식증명의 핵심 속성입니다.

관련 문서

- 03. 증명 생성 프로세스 - 검증할 증명의 생성 과정
- 05. FRI 프로토콜 - FRI 검증의 상세 내용
- 02. 회로 구성 프로세스 - 회로 구조 이해

문서 네비게이션

- **이전**: 03. 증명 생성 프로세스 - 증명 생성 방법
- **다음**: 05. FRI 프로토콜 - FRI 프로토콜 상세
- **관련 문서**:
- 01. 아키텍처 개요 - 검증자 데이터 구조
- 07. Fibonacci 예제 - 검증 실전 예제